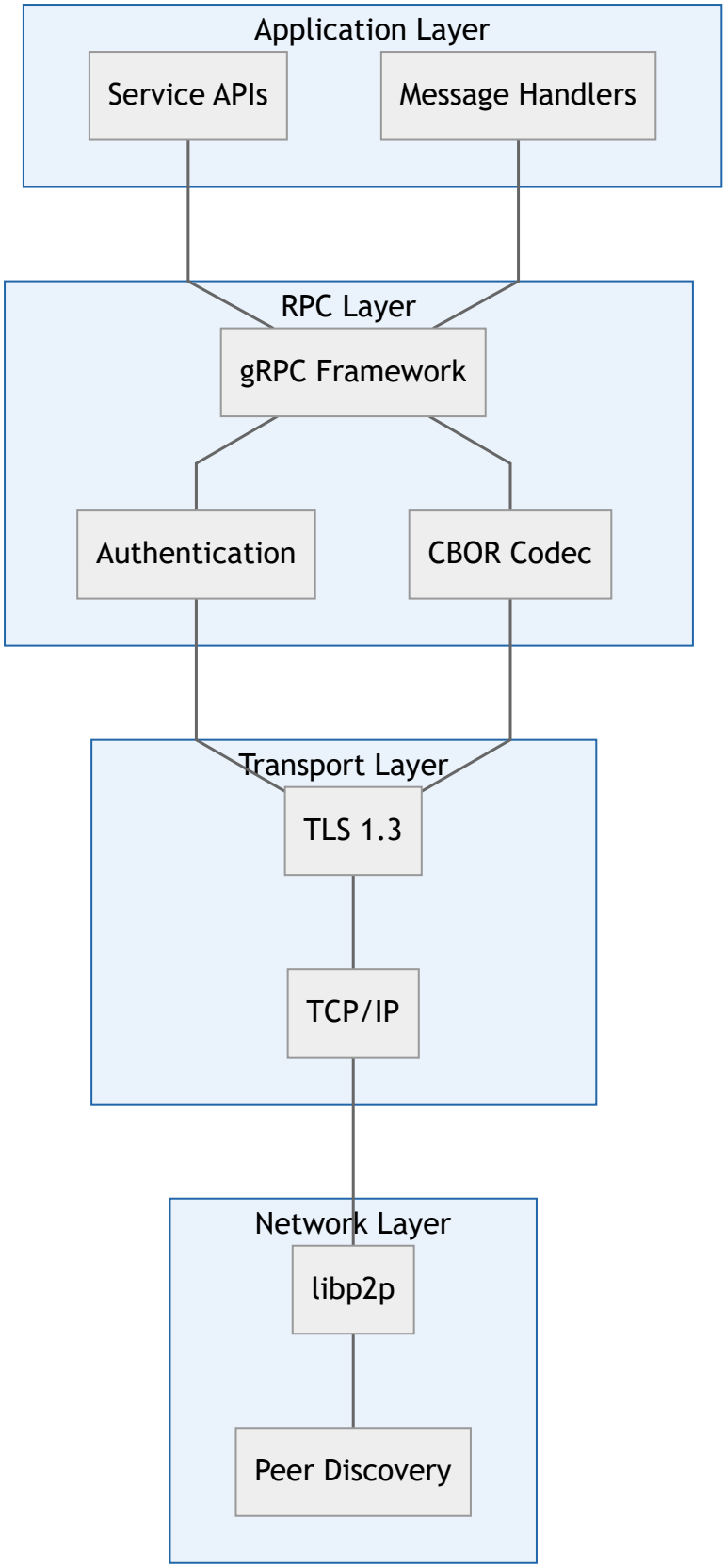
통신 프로토콜 (Communication Protocols)

소개

Oasis Core는 gRPC 기반의 통신 프로토콜을 사용하여 노드 간 안전하고 효율적인 통신을 제공합니다. 이 시스템은 인증된 gRPC 연결, CBOR 메시지 직렬화, 그리고 다양한 보안 메커니즘을 통해 신뢰할 수 있는 분산 통신을 구현합니다.

프로토콜 아키텍처

통신 스택 구조



프로토콜 계층

1. 애플리케이션 계층

- **서비스 API**: 각 모듈별 특화된 API

- **메시지 핸들러**: 요청/응답 처리 로직
- **비즈니스 로직**: 도메인 특화 처리

2. RPC 계층

- **gRPC 프레임워크**: HTTP/2 기반 RPC
- **인증 시스템**: 클라이언트 인증 및 권한 부여
- **메시지 코덱**: CBOR 직렬화/역직렬화

3. 전송 계층

- **TLS 보안**: 암호화된 통신 채널
- **TCP 연결**: 신뢰할 수 있는 데이터 전송
- **연결 관리**: 연결 풀링 및 재사용

gRPC 기반 노드 간 통신

gRPC 서버 구성

```
// gRPC 서버 설정
type ServerConfig struct {
Name  string  // 서버 이름
Port  uint16  // 포트 번호
Path  string  // 로컬 소켓 경로
Identity *identity.Identity // 노드 신원
InstallWrapper  bool  // 인터셉터 활성화
AuthFunc auth.AuthenticationFunction // 인증 함수
```

```
ClientCommonName  string  // 클라이언트 CN
CustomOptions []grpc.ServerOption // 추가 옵션
}


// gRPC 서버 생성
func  NewServer(config *ServerConfig) (*Server, error) {
// 인터셉터 체인 구성
unaryInterceptors := []grpc.UnaryServerInterceptor{
logAdapter.unaryLogger, // 로깅
serverUnaryErrorMapper, // 에러 매핑
auth.UnaryServerInterceptor(config.AuthFunc), // 인증
}
streamInterceptors := []grpc.StreamServerInterceptor{
logAdapter.streamLogger, // 로깅
serverStreamErrorMapper, // 에러 매핑
auth.StreamServerInterceptor(config.AuthFunc), // 인증
}
// 서버 옵션 설정
sOpts := []grpc.ServerOption{
grpc.ChainUnaryInterceptor(unaryInterceptors...),
grpc.ChainStreamInterceptor(streamInterceptors...),
grpc.MaxRecvMsgSize(maxRecvMsgSize),
grpc.MaxSendMsgSize(maxSendMsgSize),
```

```
grpc.KeepaliveParams(serverKeepAliveParams),
grpc.ForceServerCodec(&CBORCodec{}), // CBOR 코덱 강제 사용
}
return &Server{
server: grpc.NewServer(sOpts...),
}, nil
}
```

gRPC 클라이언트 구성

```
// gRPC 클라이언트 연결
func  Dial(target  string, opts ...grpc.DialOption) (*grpc.ClientConn, error) {
// 기본 다이얼 옵션
dialOpts := []grpc.DialOption{
grpc.WithDefaultCallOptions(
grpc.ForceCodec(&CBORCodec{}), // CBOR 코덱
grpc.MaxCallSendMsgSize(maxSendMsgSize), // 최대 송신 크기
grpc.MaxCallRecvMsgSize(maxRecvMsgSize), // 최대 수신 크기
),
grpc.WithChainUnaryInterceptor(
logAdapter.unaryClientLogger, // 클라이언트 로깅
clientUnaryErrorMapper, // 에러 매핑
),
```

```
grpc.WithChainStreamInterceptor(
logAdapter.streamClientLogger, // 스트림 로깅
clientStreamErrorMapper, // 스트림 에러 매핑
),
}
dialOpts = append(dialOpts, opts...)
return  grpc.NewClient(target, dialOpts...)
}
```

인증된 gRPC 연결

TLS 기반 상호 인증

```
// TLS 클라이언트 자격 증명 생성
func  NewClientCreds(opts *ClientOptions) (credentials.TransportCredentials, err
tlsConfig := &tls.Config{
ServerName: opts.CommonName,
// 서버 인증서 검증
VerifyPeerCertificate: func(rawCerts [][]byte, _ [][]*x509.Certificate) error {
return  cmnTLS.VerifyCertificate(rawCerts, cmnTLS.VerifyOptions{
CommonName: opts.CommonName,
Keys: opts.ServerPubKeys,
AllowUnknownKeys: false,
```

```
})
},
// 클라이언트 인증서 제공
GetClientCertificate: func(_ *tls.CertificateRequestInfo) (*tls.Certificate, err
if len(opts.Certificates) == 0 {
return nil, nil
}
return &opts.Certificates[0], nil
},
}
return credentials.NewTLS(tlsConfig), nil
}
```

공개키 기반 인증

```
// 피어 공개키 인증자
type PeerPubkeyAuthenticator struct {
sync.RWMutex
whitelist map[signature.PublicKey]bool
}

// 인증 함수
func (auth *PeerPubkeyAuthenticator) AuthFunc(ctx context.Context, _ any) error
```

```go
// 피어 정보 추출
peer, ok := peer.FromContext(ctx)
if !ok {
return  status.Errorf(codes.PermissionDenied,
"연결 피어 정보를 컨텍스트에서 가져올 수 없습니다")
}
// TLS 인증 정보 확인
tlsAuth, ok := peer.AuthInfo.(credentials.TLSInfo)
if !ok {
return  status.Errorf(codes.PermissionDenied,
"예상치 못한 피어 인증 자격 증명")
}
// 피어 인증서 검증
if  nPeerCerts := len(tlsAuth.State.PeerCertificates);  nPeerCerts != 1 {
return  status.Errorf(codes.PermissionDenied,
"예상치 못한 피어 인증서 수: %d", nPeerCerts)
}
peerCertRaw := tlsAuth.State.PeerCertificates[0].Raw
// 화이트리스트 검증
auth.RLock()
defer  auth.RUnlock()
err := cmnTLS.VerifyCertificate([][]byte{peerCertRaw}, cmnTLS.VerifyOptions{
CommonName: identity.CommonName,
```

```
Keys: auth.whitelist,
})
if  err != nil {
return  status.Errorf(codes.PermissionDenied, "%s", err.Error())
}
return  nil
}


// 공개키 허용
func (auth *PeerPubkeyAuthenticator) AllowPeerPublicKey(key signature.PublicKey)
auth.Lock()
defer  auth.Unlock()
auth.whitelist[key] = true
}
```

인증 인터셉터

```
// Unary 인증 인터셉터
func  UnaryServerInterceptor(authFunc AuthenticationFunction) grpc.UnaryServerIn
return  func(ctx context.Context,
req any,
info *grpc.UnaryServerInfo,
handler grpc.UnaryHandler,
```

```go
) (any, error) {
// 서비스별 인증 함수 확인
overrideSrv, ok := info.Server.(ServerAuth)
if !ok {
// 기본 인증 함수 사용
if err := authFunc(ctx, req); err != nil {
return nil, err
}
return handler(ctx, req)
}
// 서비스별 인증 함수 사용
if err := overrideSrv.AuthFunc(ctx, req); err != nil {
return nil, err
}
return handler(ctx, req)
}
}


// Stream 인증 인터셉터
func StreamServerInterceptor(authFunc AuthenticationFunction) grpc.StreamServer
return func(srv any,
stream grpc.ServerStream,
info *grpc.StreamServerInfo,
```

```
handler grpc.StreamHandler,
) error {
// 인증된 스트림 래퍼 생성
authStream := &authServerStream{
ServerStream: stream,
fullMethod: info.FullMethod,
authFunc: authFunc,
}
return  handler(srv, authStream)
}
}
```

메시지 직렬화 (CBOR)

CBOR 코덱 구현

```
// CBOR gRPC 코덱
type CBORCodec struct{}

// 메시지 직렬화
func (c *CBORCodec) Marshal(v any) ([]byte, error) {
return  cbor.Marshal(v), nil
}
```

```go
// 메시지 역직렬화
func (c *CBORCodec) Unmarshal(data []byte, v any) error {
return cbor.UnmarshalRPC(data, v)
}

// 코덱 이름
func (c *CBORCodec) Name() string {
return "cbor"
}
```

CBOR 사용 이유

1. **효율성**: JSON보다 작은 메시지 크기
2. **타입 안전성**: 강타입 직렬화 지원
3. **성능**: 빠른 인코딩/디코딩
4. **표준화**: RFC 8949 표준 준수

메시지 형식 예제

```go
// 요청 메시지 구조체
type Request struct {
Method string `json:"method"`
Params interface{} `json:"params,omitempty"`
RequestID uint64 `json:"id"`
```

```
}

// 응답 메시지 구조체
type Response struct {
Result  interface{} `json:"result,omitempty"`
Error *Error `json:"error,omitempty"`
ID  uint64  `json:"id"`
}

// 에러 구조체
type Error struct {
Code  int  `json:"code"`
Message  string  `json:"message"`
Data  interface{} `json:"data,omitempty"`
}
```

P2P RPC 프로토콜

RPC 서버 구현

```
// RPC 서비스 인터페이스
type Service interface {
// HandleRequest는 RPC 요청을 처리합니다
```

```
HandleRequest(ctx context.Context, method  string, body io.Reader) (io.ReadClose
}


// RPC 서버 구조체
type server struct {
Service
protocolID protocol.ID
logger *logging.Logger
}


// 스트림 핸들러
func (s *server) HandleStream(stream network.Stream) {
defer  stream.Close()
// 피어 정보 추출
peerID := stream.Conn().RemotePeer()
peerAddrInfo := peer.AddrInfo{
ID: peerID,
Addrs: []multiaddr.Multiaddr{stream.Conn().RemoteMultiaddr()},
}
// 컨텍스트에 피어 정보 추가
ctx := WithPeerAddrInfo(context.Background(), peerAddrInfo)
// 요청 읽기
var  req Request
```

```go
if err := cbor.NewDecoder(stream).Decode(&req); err != nil {
s.logger.Error("요청 디코딩 실패", "err", err)
return
}
// 요청 처리
respReader, err := s.HandleRequest(ctx, req.Method, bytes.NewReader(req.Body))
if err != nil {
// 에러 응답 전송
resp := Response{
Error: &Error{
Code: -1,
Message: err.Error(),
},
ID: req.ID,
}
_ = cbor.NewEncoder(stream).Encode(resp)
return
}
defer respReader.Close()
// 성공 응답 전송
respBody, _ := io.ReadAll(respReader)
resp := Response{
Result: respBody,
```

```
ID: req.ID,
}
_ = cbor.NewEncoder(stream).Encode(resp)
}
```

RPC 클라이언트 구현

```
// RPC 클라이언트 구조체
type client struct {
host host.Host
protocolID protocol.ID
logger *logging.Logger
}

// RPC 호출
func (c *client) Call(ctx context.Context, peerID peer.ID, method  string, reque
// 피어에 연결
stream, err := c.host.NewStream(ctx, peerID, c.protocolID)
if  err != nil {
return  nil, fmt.Errorf("스트림 생성 실패: %w", err)
}
// 요청 준비
reqBody, err := io.ReadAll(request)
```

```go
if err != nil {
stream.Close()
return nil, fmt.Errorf("요청 읽기 실패: %w", err)
}
req := Request{
Method: method,
Body: reqBody,
ID: generateRequestID(),
}
// 요청 전송
if err := cbor.NewEncoder(stream).Encode(req); err != nil {
stream.Close()
return nil, fmt.Errorf("요청 전송 실패: %w", err)
}
// 응답 수신
var resp Response
if err := cbor.NewDecoder(stream).Decode(&resp); err != nil {
stream.Close()
return nil, fmt.Errorf("응답 수신 실패: %w", err)
}
stream.Close()
// 에러 확인
if resp.Error != nil {
```

```
return nil, fmt.Errorf("RPC 에러: %s", resp.Error.Message)
}
return io.NopCloser(bytes.NewReader(resp.Result.([]byte))), nil
}
```

프로토콜 버퍼 사용법

서비스 정의 예제

```
// storage.proto - 스토리지 서비스 정의
syntax = "proto3";

package oasis.storage.v1;

// 스토리지 서비스
service Storage {
// 키-값 쌍 조회
rpc Get(GetRequest) returns (GetResponse);
// 키-값 쌍 저장
rpc Put(PutRequest) returns (PutResponse);
// 키 삭제
rpc Delete(DeleteRequest) returns (DeleteResponse);
// 반복자 생성
```

```
rpc  GetPrefixes(GetPrefixesRequest) returns (stream GetPrefixesResponse);
}

// 조회 요청
message GetRequest {
bytes  key = 1;
uint64  round = 2;
}

// 조회 응답
message GetResponse {
bytes  value = 1;
bytes  proof = 2;
}

// 저장 요청
message PutRequest {
bytes  key = 1;
bytes  value = 2;
}

// 저장 응답
message PutResponse {
```

```
bytes  receipt = 1;
}
```

gRPC 서비스 등록

```
// 스토리지 서비스 구현
type storageService struct {
backend storage.Backend
}

// Get 메서드 구현
func (s *storageService) Get(ctx context.Context, req *GetRequest) (*GetResponse
value, err := s.backend.Get(ctx, &storage.GetRequest{
Tree: storage.TreeID{
Root: req.Root,
Version: req.Round,
},
Key: req.Key,
})
if  err != nil {
return  nil, err
}
return &GetResponse{
```

```
Value: value.Value,
Proof: value.Proof,
}, nil
}

// 서비스 등록
func RegisterStorageService(server *grpc.Server, backend storage.Backend) {
service := &storageService{backend: backend}
RegisterStorageServer(server, service)
}
```

통신 프로토콜 API

서비스 등록 및 관리

```
// 서비스 등록 인터페이스
type ServiceRegistry interface {
// RegisterService는 gRPC 서비스를 등록합니다
RegisterService(desc *grpc.ServiceDesc, impl interface{})
// RegisterProtocolServer는 P2P 프로토콜 서버를 등록합니다
RegisterProtocolServer(srv rpc.Server)
// RegisterHandler는 메시지 핸들러를 등록합니다
RegisterHandler(topic string, handler Handler)
```

```
}

// 메시지 핸들러 인터페이스
type Handler interface {
// DecodeMessage는 수신된 메시지를 디코딩합니다
DecodeMessage(msg []byte) (any, error)
// AuthorizeMessage는 메시지 권한을 확인합니다
AuthorizeMessage(ctx context.Context, peerID signature.PublicKey, msg any) error
// HandleMessage는 메시지를 처리합니다
HandleMessage(ctx context.Context, peerID signature.PublicKey, msg any, isOwn  b
}
```

클라이언트 API

```
// gRPC 클라이언트 생성
func  NewStorageClient(conn *grpc.ClientConn) StorageClient {
return &storageClient{cc: conn}
}

// P2P RPC 클라이언트 생성
func  NewP2PClient(host host.Host, protocolID protocol.ID) rpc.Client {
return  rpc.NewClient(host, protocolID)
}
```

```go
// 통합 클라이언트 인터페이스
type Client interface {
// gRPC 호출
CallGRPC(ctx context.Context, method  string, req, resp  interface{}) error
// P2P RPC 호출
CallP2P(ctx context.Context, peerID peer.ID, method  string, req io.Reader) (io.
// 메시지 발행
Publish(ctx context.Context, topic  string, msg  interface{})
}
```

클라이언트 예제

gRPC 클라이언트 사용

```go
// gRPC 클라이언트 연결 및 사용
func  useGRPCClient() error {
// TLS 자격 증명 생성
creds, err := grpc.NewClientCreds(&grpc.ClientOptions{
CommonName: "oasis-node",
ServerPubKeys: map[signature.PublicKey]bool{
serverPubKey: true,
},
```

```go
Certificates: []tls.Certificate{clientCert},
})
if err != nil {
return fmt.Errorf("TLS 자격 증명 생성 실패: %w", err)
}
// gRPC 연결
conn, err := grpc.Dial("localhost:9001", grpc.WithTransportCredentials(creds))
if err != nil {
return fmt.Errorf("gRPC 연결 실패: %w", err)
}
defer conn.Close()
// 클라이언트 생성
client := NewStorageClient(conn)
// API 호출
resp, err := client.Get(context.Background(), &GetRequest{
Key: []byte("test-key"),
Round: 100,
})
if err != nil {
return fmt.Errorf("Get 호출 실패: %w", err)
}
fmt.Printf("값: %s\n", string(resp.Value))
return nil
```

```
}
```

P2P RPC 클라이언트 사용

```
// P2P RPC 클라이언트 사용
func usePP2PClient() error {
// P2P 호스트 생성
host, err := libp2p.New()
if err != nil {
return fmt.Errorf("P2P 호스트 생성 실패: %w", err)
}
defer host.Close()
// RPC 클라이언트 생성
client := rpc.NewClient(host, "/oasis/storage/sync/1.0.0")
// 요청 데이터 준비
req := &SyncRequest{
Root: rootHash,
Leaves: leafHashes,
}
reqData := bytes.NewReader(cbor.Marshal(req))
// RPC 호출
respReader, err := client.Call(context.Background(), targetPeerID, "Sync", reqDa
if err != nil {
```

```go
return fmt.Errorf("RPC 호출 실패: %w", err)
}
defer respReader.Close()
// 응답 처리
var resp SyncResponse
respData, _ := io.ReadAll(respReader)
if err := cbor.Unmarshal(respData, &resp); err != nil {
return fmt.Errorf("응답 디코딩 실패: %w", err)
}
fmt.Printf("동기화된 노드 수: %d\n", len(resp.Nodes))
return nil
}
```

메시지 핸들러 구현

```go
// 커스텀 메시지 핸들러
type CustomHandler struct {
logger *logging.Logger
}

// 메시지 디코딩
func (h *CustomHandler) DecodeMessage(msg []byte) (any, error) {
var message CustomMessage
```

```go
if  err := cbor.Unmarshal(msg, &message);  err != nil {
return  nil, fmt.Errorf("메시지 디코딩 실패: %w", err)
}
return &message, nil
}


// 메시지 권한 확인
func (h *CustomHandler) AuthorizeMessage(ctx context.Context, peerID signature.P
message, ok := msg.(*CustomMessage)
if !ok {
return  fmt.Errorf("잘못된 메시지 타입")
}
// 메시지 서명 검증
if !peerID.Verify(message.Signature, message.Data) {
return  fmt.Errorf("메시지 서명 검증 실패")
}
return  nil
}


// 메시지 처리
func (h *CustomHandler) HandleMessage(ctx context.Context, peerID signature.Publ
if  isOwn {
// 자신이 발송한 메시지는 무시
```

```
return nil
}
message := msg.(*CustomMessage)
h.logger.Info("메시지 수신",
"peer_id", peerID,
"message_type", message.Type,
"data_size", len(message.Data),
)
// 메시지 처리 로직
return h.processMessage(message)
}

// 핸들러 등록
func registerCustomHandler(p2p api.Service) {
handler := &CustomHandler{
logger: logging.GetLogger("custom-handler"),
}
topic := "custom-topic"
p2p.RegisterHandler(topic, handler)
}
```

보안 설정

TLS 설정

```yaml
# TLS 보안 설정
grpc:
# 서버 TLS 설정
server:
cert_file: "/path/to/server.crt"
key_file: "/path/to/server.key"
ca_file: "/path/to/ca.crt"
# 클라이언트 인증 요구
client_auth: require
# 지원하는 TLS 버전
min_version: "1.3"
max_version: "1.3"
# 암호화 스위트
cipher_suites:
- "TLS_AES_256_GCM_SHA384"
- "TLS_CHACHA20_POLY1305_SHA256"
# 클라이언트 TLS 설정
client:
cert_file: "/path/to/client.crt"
key_file: "/path/to/client.key"
ca_file: "/path/to/ca.crt"
```

```yaml
# 서버 이름 검증
server_name: "oasis-node"
# 인증서 검증 비활성화 (개발용만)
insecure_skip_verify: false
```

인증 설정

```go
// 다중 인증 방식 설정
func setupAuthentication() auth.AuthenticationFunction {
return func(ctx context.Context, req any) error {
// 1. TLS 클라이언트 인증서 확인
if err := checkClientCertificate(ctx); err != nil {
return err
}
// 2. API 키 확인
if err := checkAPIKey(ctx); err != nil {
return err
}
// 3. 요청 서명 확인
if err := checkRequestSignature(ctx, req); err != nil {
return err
}
return nil
```

```
}

}
```

성능 최적화

연결 풀링

```
// gRPC 연결 풀 설정

type ConnectionPool struct {

sync.RWMutex

connections  map[string]*grpc.ClientConn

maxConns  int

}


// 연결 가져오기

func (p *ConnectionPool) GetConnection(target  string) (*grpc.ClientConn, error)

p.RLock()

conn, exists := p.connections[target]

p.RUnlock()

if  exists && conn.GetState() == connectivity.Ready {

return  conn, nil

}

// 새 연결 생성
```

```go
p.Lock()
defer p.Unlock()
if len(p.connections) >= p.maxConns {
return nil, fmt.Errorf("연결 풀 한계 도달")
}
conn, err := grpc.Dial(target, grpc.WithTransportCredentials(insecure.NewCredent
if err != nil {
return nil, err
}
p.connections[target] = conn
return conn, nil
}
```

메시지 압축

```go
// gRPC 압축 설정
func setupCompression() []grpc.ServerOption {
return []grpc.ServerOption{
grpc.RPCCompressor(grpc.NewGZIPCompressor()),
grpc.RPCDecompressor(grpc.NewGZIPDecompressor()),
}
}
```

```go
// 클라이언트 압축 사용
func callWithCompression(client StorageClient) error {
ctx := context.Background()
resp, err := client.Get(ctx, &GetRequest{
Key: []byte("large-data-key"),
}, grpc.UseCompressor(gzip.Name))
return err
}
```

모니터링 및 로깅

메트릭 수집

```go
// gRPC 메트릭 정의
var (
grpcServerCalls = prometheus.NewCounterVec(
prometheus.CounterOpts{
Name: "oasis_grpc_server_calls_total",
Help: "gRPC 서버 호출 수",
},
[]string{"method", "status"},
)
grpcServerLatency = prometheus.NewHistogramVec(
```

```
prometheus.HistogramOpts{
Name: "oasis_grpc_server_duration_seconds",
Help: "gRPC 서버 응답 시간",
},
[]string{"method"},
)
)

// 메트릭 인터셉터
func metricsUnaryInterceptor(ctx context.Context, req any, info *grpc.UnaryServ
start := time.Now()
resp, err := handler(ctx, req)
status := "success"
if err != nil {
status = "error"
}
grpcServerCalls.WithLabelValues(info.FullMethod, status).Inc()
grpcServerLatency.WithLabelValues(info.FullMethod).Observe(time.Since(start).Sec
return resp, err
}
```

구조화된 로깅

```go
// 로깅 인터셉터
func loggingUnaryInterceptor(ctx context.Context, req any, info *grpc.UnaryServ
logger := logging.GetLogger("grpc-server")
// 요청 로깅
logger.Debug("gRPC 요청 수신",
"method", info.FullMethod,
"request", req,
)
start := time.Now()
resp, err := handler(ctx, req)
duration := time.Since(start)
// 응답 로깅
if err != nil {
logger.Error("gRPC 요청 실패",
"method", info.FullMethod,
"duration", duration,
"error", err,
)
} else {
logger.Info("gRPC 요청 성공",
"method", info.FullMethod,
"duration", duration,
```

```
)
}
return  resp, err
}
```

문제 해결

일반적인 문제

1. **연결 실패**

```
# 네트워크 연결 확인
telnet target-host 9001
# TLS 핸드셰이크 테스트
openssl s_client -connect target-host:9001 -cert client.crt -key client.key
```

2. **인증 실패**

```
# 인증서 유효성 확인
openssl x509 -in client.crt -text -noout
# 공개키 확인
oasis-node identity show-tls-pubkey --datadir /path/to/datadir
```

3. **성능 문제**

```
# gRPC 메트릭 확인
```

```
curl http://localhost:9090/metrics | grep  grpc

# 연결 상태 확인

grpcurl -plaintext localhost:9001 grpc.health.v1.Health/Check
```

디버깅 도구

```
# gRPC 서비스 목록 조회

grpcurl  -plaintext  localhost:9001  list


# 메서드 호출 테스트

grpcurl  -plaintext  -d  '{"key": "dGVzdA=="}'  localhost:9001  oasis.storage.v1


# 연결 상태 모니터링

grpcurl  -plaintext  localhost:9001  grpc.reflection.v1alpha.ServerReflection/Se
```

이 문서는 Oasis Core의 통신 프로토콜에 대한 포괄적인 가이드를 제공합니다. gRPC 기반의 안전하고 효율적인 노드 간 통신을 통해 분산 블록체인 네트워크의 신뢰성과 성능을 보장할 수 있습니다.